COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI 682022

DEPARTMENT OF COMPUTER APPLICATIONS

PROJECT REPORT

DETECTION OF TUBERCULOSIS BACILLI USING DEEP LEARNING

Submitted by

Kiran S R
Abhiram S
Arifa Nazrin N
Aldrin Martin Simenthy
Sabir Ahmad Sabiri

in partial fulfillment for the award of the degree of

Master of Computer Applications
May 2022

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI 682 022



CERTIFICATE

This is to certify that this project report entitled "**DETECTION OF TUBERCULOSIS BACILLI USING DEEP LEARNING**" *is a bonafide record of work done by* "**Kiran S R, Abhiram S, Arifa Nazrin N, Aldrin Martin Simenthy, Sabir Ahmad Sabiri**" *towards the partial fulfillment of the requirements for the award of Master of Computer Application degree at Department of Computer Application, Cochin University of Science and Technology*.

SIGNATURE	**SIGNATURE**
Dr. M V Judy	Dr. Vishnukumar S
HEAD OF THE DEPARTMENT	**Associate Professor**
Department of Computer Applications	Department of Computer Applications
CUSAT	CUSAT

DECLARATION

We KIRAN S R (Register Number : 38119032), ABHIRAM S (Register Number : 38119004), ARIFA NAZRIN N (Register Number :38119017), ALDRIN MARTIN SIMENTHY (Register Number : 38119008), SABIR AHMAD SABIRI (Register Number : 38119047) hereby declare that the Report entitled "**DETECTION OF TUBERCULOSIS BACILLI USING DEEP LEARNING**" in partial fulfillment of the requirements for the award of the Degree of Master of Computer Application is a record of bonafide work done by me during the period from January 2022 to May 2022 under the supervision and guidance of Bineetha Vijayan at Department of Computer Applications , CUSAT.

Signature of the Candidates

ACKNOWLEDGEMENT

With great pleasure we hereby acknowledge that the help given to by various individuals throughout the project itself is an acknowledgement to the inspiration driven and technical assistance contributed by many individuals.

We extend our sincere thanks to all the non-teaching staff for providing the necessary facilities and help. We thank god, almighty for his immeasurable blessing upon my life.

We are pleased with our indebtedness to Dr. M V Judy, Head of the Department, Department of Computer Applications. CUSAT for him. gracious encouragement.

And also we are thankful to Dr. Vishnukumar S, our project guide for beloved support. We are obliged to the teaching staff for being helpful and cooperative during the period of the project. We extend our heartfelt thanks to parents, friends and well wishes for their support and timely help.

ABSTRACT

Diagnosis of tuberculosis (TB) using manual observation does not work well because it is time consuming and the effectiveness depends on the pathologist and the smear level. Tuberculosis (TB) is an infectious disease caused by Mycobacterium tuberculosis.. Most low- and middle-income countries use the image of a small sputum to detect bacilli leading to TB. Manual testing will reduce accuracy and take longer. To avoid these, we propose a deep learning model with the objectives of detecting TB, single bacilli or overlapping bacilli and the severity of the TB. The model uses patched images of bacilli. The contrast of these images are enhanced and mask slic segmentation is performed. The obtained image is then given as input to a custom CNN. The model returned an accuracy of 94.62%.

TABLE OF CONTENTS

1. INTRODUCTION 7

1.1 PROJECT OBJECTIVES 8

1.2 PROJECT SYSTEM 8

2. SYSTEM REQUIREMENTS 9

2.1 HARDWARE REQUIREMENTS 9

2.2 SOFTWARE REQUIREMENTS 9

3. LITERATURE SURVEY 12

4. METHODOLOGY OF SYSTEM 15

4.1 SYSTEM ARCHITECTURE 15

4.2 IMPLEMENTATION 16

5. RESULT AND ANALYSIS 24

6. CONCLUSION 26

7. SCOPE OF FUTURE DEVELOPMENT 27

8. BIBLIOGRAPHY 28

9. APPENDIX 30

CHAPTER I
INTRODUCTION

TB is a serious infectious disease that mainly affects the lungs which accounts for a major global public health problem worldwide. It is a chronic infectious disease caused by a virus called Mycobacterium tuberculosis. Bacteria often infect the lungs but can also damage other parts of the human body. TB spreads through the air, when a person coughs, in this way, the spread of the disease. It can be prevented and treated if detected early; otherwise, it may result in the death of the patient. Tests such as a chest X-ray or a sputum sample can be performed to determine if a person has tuberculosis. According to the World Health Organization, tuberculosis is one of the ten most common causes of cough, and you can get tuberculosis by inhaling droplets of air or by coughing or sneezing from an infected person called primary tuberculosis (TB) which is the leading cause of death in the world.

This project gives an automatic method for the detection of Tuberculosis (TB) bacilli from microscopic sputum smear images. The World Health Organization (WHO) has declared tuberculosis is a global emergency. It is estimated that around one-third of the World's population is infected with mycobacterium tuberculosis.

Tuberculosis is a contagious disease where early detection is important in controlling the disease. Conventional microscopy is the most common technique used in the routine diagnosis of TB. In this method, the diagnosis of tubercle bacilli is usually performed on sputum smears using a microscope, which is used extensively to diagnose sputum smears found in suspected patients.

TB is curable if detected early. There are a number of limitations associated with practice that include human error and fatigue, lack of medical facilities, and low accuracy. Computer-assisted tools helped to reduce these limitations. Calculation

methods take pictures and use supervised / unsupervised learning methods to identify bacilli correctly.

1.1 PROJECT OBJECTIVES

The main objectives of developing this project are detection of tuberculosis bacteria and automating the detection method.

1.2 PROPOSED SYSTEM

The aim of the proposed is to detect tuberculosis bacilli from sputum smear images using convolutional neural networks. We collect dataset of sputum smear images and enhance the image to improve the image quality by increasing sharpness, brightness, pixel density, and other characteristics. Then we train using convolutional neural network model. Before training it undergoes contrast enhancement and segmentation then fed into the model. which classifies them as bacilli or non-bacilli.

CHAPTER II
SYSTEM REQUIREMENTS

The system and software requirements for the project are mentioned in the following sections.

2.1 HARDWARE REQUIREMENTS

- RAM: 16 GB RAM
- PROCESSOR: Intel Core i7 CPU
- DISPLAY: 720 Display
- HARD DISK: 1TB
- PROCESSOR SPEED: 2Ghz

2.2 SOFTWARE REQUIREMENTS

- **Windows-10**

  Microsoft's Windows 10 is a set of programmes that were introduced as part of the Windows NT family of apps. Windows 10 is now available for download from MSDN and Technet, as well as as a free upgrade for business Windows 8 copies from the Windows Store. In addition to building additional tests for Windows 10, which are available on Windows Insider, Windows 10 receives ongoing new builds at no further cost to customers. During their ten years of extended support life, business-enabled devices may receive these updates at a reduced rate or utilize a maintenance history that receives only significant upgrades, such as security episodes.

- **Jupyter notebook**

  Jupyter is a non-profit dedicated to "developing open source software, open standards, and interactive computer services in all programming languages." The Fernando Pérez project Jupyter released IPython in 2014, which allows working conditions in a variety of languages. Pandas Library is a library dedicated to pandas. In computer programming, pandas is a data processing and analysis software package built for the Python programming language. It includes data structures and procedures for manipulating numerical tables and time series in particular. Free software distributed under the terms of a three-phase BSD license. The word "panel data" is an econometrics term for data sets that have visibility in the majority of the same people.

- **Tensorflow & keras library**

  TensorFlow is a deep reading framework developed by Google at the end of the open source era and published in 2015. TensorFlow is a mathematical library for creating emotional networks that is ideal for coordinating data flow across multiple tasks. A large selection of production and training models are available. TensorFlow is a flexible and extensive ecosystem for public resources, libraries, and tools that assist in the development and distribution of machine learning applications.

  Keras is a Python-based high-level neural network with an Application Programming Interface (API). This open source network neural library works with CNTK, TensorFlow, and Theano to offer quick testing of deep neural networks. It doesn't keep low statistics; instead, it sends them to Backend, another library.

  Keras was adopted and integrated into TensorFlow in mid-2017. Users can access it via the tf.keras module. However, the Camera library can still operate separately and independently.

- **Open CV**

  OpenCV (Open Source Computer Vision Library) is a computer and software library that is free to use. OpenCV is intended to provide a comprehensive infrastructure for computer vision applications, as well as to speed up the incorporation of machine vision into commercial solutions. OpenCV makes it simple for businesses to utilize and alter code because it is a licensed BSD product.

  OpenCV is a large open-source library for image processing, computer vision, and machine learning. Python, C++, Java, and other programming languages are supported by OpenCV. It can analyze photos and movies to recognise items, faces, and even human handwriting. When it's paired with other libraries, such as Numpy, a highly efficient library for numerical operations, the number of weapons in your arsenal grows, i.e., every operation that Numpy can perform can be merged with OpenCV.

- **Scikit_image**

  Scikit-image is a Python library that works with NumPy arrays and is open source. It makes use of algorithms and resources in research, education, and industry. Even for newbies to the Python ecosystem, it is a simple and straightforward library. The code is of excellent quality, has been peer-reviewed, and was produced by a volunteer community.

- **Numpy**

  NumPy is the most important Python package for scientific computing. It's a Python library that includes a multidimensional array object, derived objects (such as masked arrays and matrices), and a variety of routines for performing fast array operations, such as mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation, and more.

CHAPTER III
LITERATURE REVIEW

The detection of tuberculosis bacilli in small sputum samples is an emerging area of research. There are only a limited number of research papers available for this topic.

Three CNN models were built in the study by Yadini Perez Lopez, C.F.F Costa Filho, L.M.R Aguilera, M.G.F. Costa [1], with the first model having one convolutional layer, the second with one extra layer, and the third with one extra layer than the second layer. They used two datasets to train these models. The first was the original dataset, which includes patches with and without bacilli retrieved from the original dataset. A greyscale and (R-G) version of the dataset is also produced. The second one has patches that have been regularized, as well as grayscale and (R-G) versions. The models were developed using various datasets, and the results were obtained. The third model was 99 percent accurate in predicting the existence of bacilli.

Sonaal Kant and Muktabh Mayank Srivastava reported a model with 83.78 percent recall for detecting TB bacilli in test photos in their paper [2]. A five-layered convolutional layer is introduced in this paper. The results demonstrate that the model was skewed toward positive situations, resulting in a high number of false positives. As a result, a new model was created and utilized to classify the false positives. The first model's suspect is given to a new model, which determines if the suspect is genuine or false positive. The patches are used to train this model. With cascading and class weights, two models were trained. To limit the number of false positives, class weights are utilized. The precision of this model is 67.55, while the f1 score is 74.79.

Rani Oomman Panicker, Kaushik S. Kalmady, Jeny Rajan, M.K. Sabu [3], the dataset is first pre-processed. Denoising was part of the image preprocessing, and it was then converted to grayscale. After that, Otsu thresholding is used to segment it. Then comes morphological opening and closing. Following that, connected component analysis is used to extract all connected components, which are then sent into the second stage, which determines whether or not the connected component is a bacilli. The final CNN is made up of two convolutional layers with 32 (3x3) filter maps, three convolutional layers with 64 (3x3) filters, and one convolutional layer with 128 (3x3) filters.. The output from this series of convolution layers is fed to a fully connected layer with 128 neurons with Rectified Linear Unit (ReLU) activation and finally a sigmoid neuron for binary classification. The proposed model was trained with patches of size 224x224. The layers were added with each training. The proposed method had a sensitivity of 97.13%.

Mohammad I Shah, Smriti Mishra, Malay Sarkar, Chittaranjan Rout [4], The first dataset is transformed to grayscale, and then the contrast is enhanced. After that, thresholding is used to convert grayscale to binary. After the removal of huge objects and artifacts. Following that, the datasets are segmented using watershed segmentation. The bacilli are then labeled and counted.

In [5], The research examines the need for image pre-processing prior to training. They convert RGB to HSV format in this study, then identify two classes, foreground and background, and adjust hue and saturation automatically. The image is then transformed to RGB once more. Contrast enhancement is the term for this process. This leads to more accurate predictions. Without pre-processing, prediction accuracy ranges from 20% to 30%, but with pre-processing, the model achieves about 100% accuracy. The neural network's training time was also lowered as a result of this.

A model proposed in paper [6] Diagnosis and Counting of Tuberculosis Bacilli Using Digital Image Processing by Yoges Payasi and Savitanandan Patidar (2017), they first convert the RGB image to HSI which is better for preprocessing. After that they convert the image into grayscale and threshold giving a particular value. Then the noise is removed using some function then again the image is segmented. Then the area perimeter all is identified of bacilli to get a size and shape characteristics to identify the bacilli.Specification satisfying above conditions are counted. The results show the model has 90% of accuracy.

In a paper [7] by K. S. Mithra, W. R. Sam Emmanuel, they proposed deep learning which is trained with extracted features of bacilli. First they reduce noise using adaptive median filtering and the intensity modification is done. Green channel segmentation followed by area segmentation is done. Location-oriented histogram(LoH) and Speed up robust feature (SURF) are used to extract features. Then these extracted features are trained using deep neural networks. Bacilli identification is performed using the CAT algorithm. The system was successfully tested for disease identification which attained accuracy, sensitivity, and specificity of about 97.55, 97.86 and 98.23 on sputum image dataset.

CHAPTER IV
METHODOLOGY

Proposed method

In this project we propose a system which contains two stages. The initial stage consists of image preprocessing and image segmentation. For image preprocessing, we used contrast enhancement to highlight the details in the image followed by an image segmentation process by using masked-simple linear iterative clustering. The second stage is a Convolutional Neural Network(CNN), which consists of 6 layers and 2 dense layers. we use cropped bacilli images of size 224 x 224, to train the CNN model. Before training the model the patch images are preprocessed and segmented. For classification and prediction purposes we use the original image and do preprocessing and segmentation and finally the result is fed into the model.

4.1 SYSTEM ARCHITECTURE

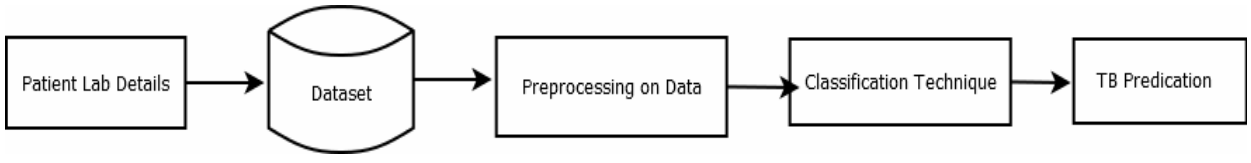


Fig (4.1)

4.2 IMPLEMENTATION

4.2.1 IMAGE PRE PROCESSING :

In this project we use the contrast enhancement as the image pre-processing. For that we use morphological transformation functions from openCV.

Image pre-processing may have a significant positive impact on the quality of feature extraction and image analysis outcomes. The mathematical normalization of a data collection, which is a typical step in many feature descriptor approaches, is akin to image pre-processing. Working with neural networks and deep learning models necessitates data preparation. Image processing is a technique for applying operations on an image in order to improve it or extract relevant information from it. Image processing is one of the most quickly evolving technologies today. It is also a major research topic in the fields of engineering and computer science.

The sputum smear contains a lot of mucus, contaminants, and necrotic tissues, therefore different sections of the same image in the microscopic imaging process will have varying sensitivity to light intensity. To achieve intelligent bacilli identification, a large dynamic range of the microscopic picture is required to gather all of the characteristic information of the sputum smear detection zone. In this process used functions are morphological gradient, structuring element.

Image enhancement can improve the image quality by increasing sharpness, brightness, pixel density, and other characteristics. It mostly relies on two factors: time and frequency. Consider a basic bar graph with the x-axis (horizontal) representing time and the y-axis representing frequency (vertical). The enhancement is carried out and altered based on these two main components.

4.2.2 SEGMENTATION :

In our project we use masked simple linear iterative clustering (m-slic) segmentation process.

To improve visibility, contrast enhancement procedures modify the relative brightness and darkness of items in the scene. A gray-level transform can be used to adjust the image's contrast and tone by mapping the image's gray levels to new values. By increasing the brightness difference between items and their backgrounds, contrast enhancements improve the perceptibility of objects in the scene. Contrast enhancements are usually done in two steps: a contrast stretch and a tonal enhancement, though they can be done simultaneously. Tonal enhancements improve the brightness differences in the shadow (dark), midtone (grays), or highlight (bright) regions at the expense of the brightness differences in the other regions, whereas a contrast stretch improves the brightness differences uniformly across the dynamic range of the image.

Accurate nucleus and cell segmentations are critical for improving target recognition accuracy in microscopic cell pictures. An enhanced SLIC superpixel segmentation algorithm based on grayscale enhancement and regional equalization is developed because the traditional SLIC (Simple Linear Iterative Clustering) approach cannot segment microscopic cell images well. The best classification multi-threshold method based on maximum entropy criterion is used to nonlinearly enhance the gray scale of the original images, while also greatly improving the balance of each classification region, based on the characteristics of microscopic cell images, selecting different transformation parameters with the conditional iterative algorithm.To achieve superpixel segmentation of the image, the gray distance and spatial distance are calculated in the circle neighborhood of the cluster center, respectively. Finally, the comparison algorithm and the revised SLIC method are analyzed and tested. Our revised SLIC algorithm model has higher segmentation accuracy and is more suited for cell segmentation in microscopic cell images than the original SLIC algorithm, according to the experimental results.

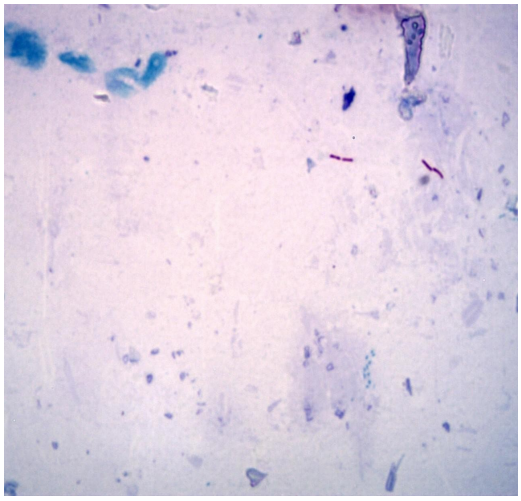
$$d_{rgb} = \sqrt{(R_k - R_i)^2 + (G_k - G_i)^2 + (B_k - B_i)^2}$$

$$d_{xy} = \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}$$

$$D_s = d_{rgb} + \frac{m}{S} * d_{xy}$$

All the pixels are associated with the nearest cluster center, a new center is computed as the average [R, G, B, x, y] vector of all the pixels belonging to the cluster. Which gives the values of RGB components of the image along with its x and y coordinates. S in the equation gives the details of super pixels in the image. This process of associating pixels with the nearest cluster center and recomputing the center is iteratively repeated until convergence

.



(a)

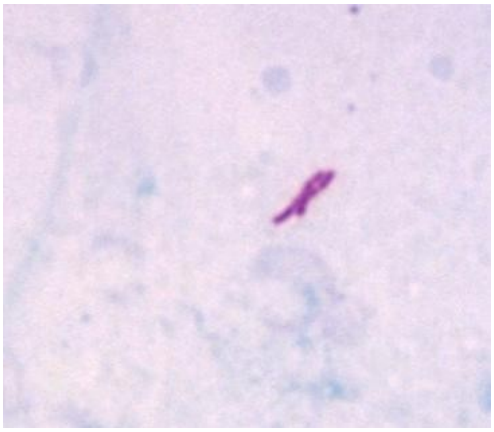


(b)



(c) (d)

Fig (4.2) a): Original image b): Mask image c): Inverted image d): Segmented image





Fig(4.3) a): patch images b): Segmented patch image

4.2.3 CLASSIFICATION :

In deep learning, convolutional neural networks (CNN or ConvNet) are the most commonly used class of deep neural networks for the analysis of visual images. CNN is a regulated version of the Multilayer Perceptron. Multilayer perceptron usually means a fully connected network. That is, all neurons in one layer are connected to all neurons in the next layer. The "full connectivity" of these networks can make them vulnerable to oversized data. Common ways to normalize or prevent overfitting are parameter penalties during exercise (e.g. weight loss) or reduced connectivity (skipping connections, dropouts, etc.). CNN will take a different approach to regularization. Take advantage of hierarchical patterns of data and use smaller, simpler patterns embedded in filters to build more complex patterns. As a result, CNNs are not extreme in terms of connectivity and complexity.

Convolutional networks were inspired by biological processes because the patterns of connections between neurons resemble the tissues of the animal's visual cortex. Individual cortical neurons respond to stimuli only in a limited area of the visual field known as the receptive field. The receptive fields of different neurons partially overlap and thus cover the entire visual field.

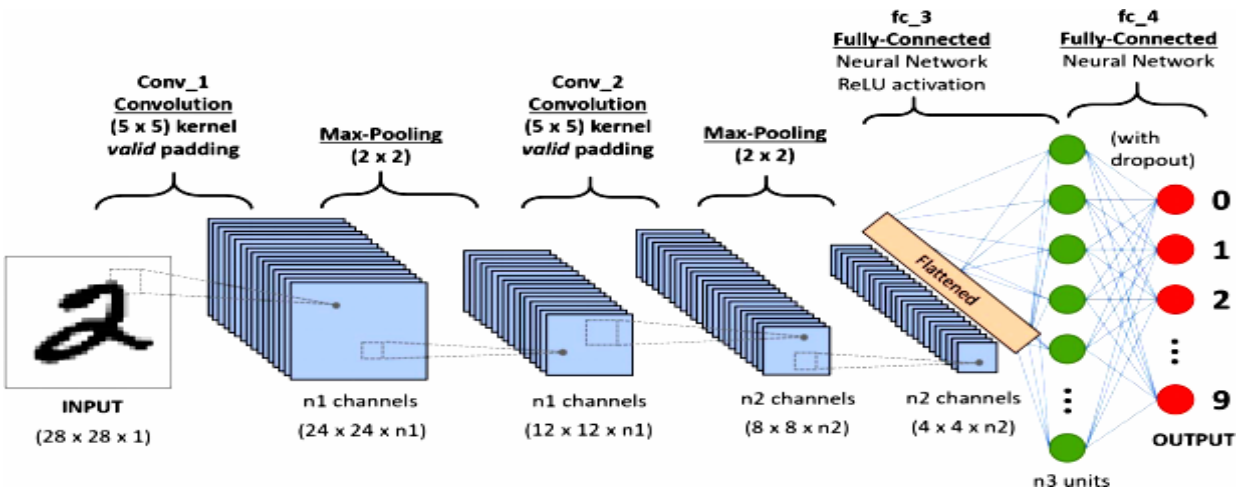
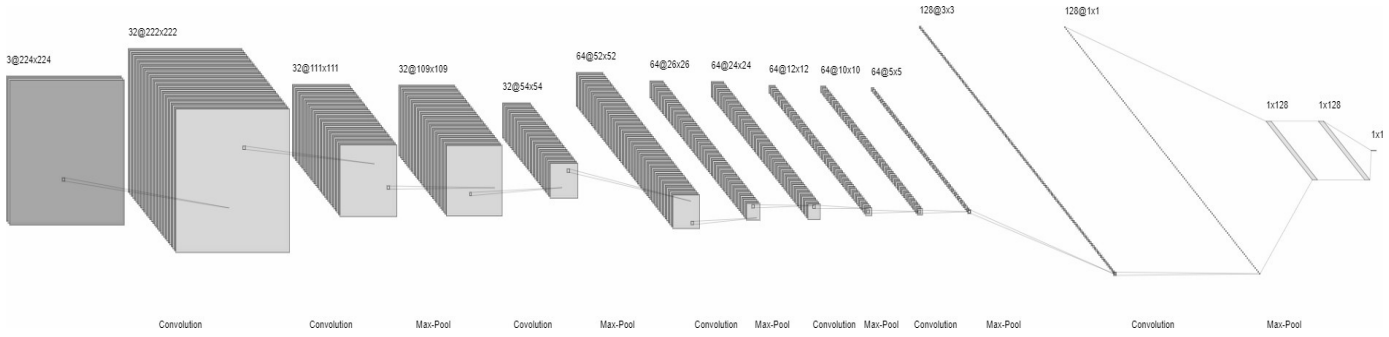


Fig (4.3)

CNN uses relatively little preprocessing compared to other image classification algorithms. This means that while networks learn filter (or kernel) optimization through automatic learning, traditional algorithms develop these filters manually. This independence from previous knowledge and human intervention in feature extraction is a great advantage.

In our proposed method we take patch images of bacilli and non bacilli about 400 images. Did augmentation on the patch dataset and finally got a dataset of 4000 patch images. Which is used later to train the CNN model. Before training it undergoes contrast enhancement and segmentation then fed into the model. In the second phase the whole image is taken and does the preprocessing and detects the area of interest. And finally these identified objects are fed into the trained model for prediction and classification as bacilli and non bacilli. The model consists of 6 layers and in the final layer we applied a sigmoid function. The filter size of (5x5) is used. Padding used is 'same' and the stride is 1 in our model. A total number of epochs is 25.



Fig(4.4) CNN Model

4.2.4 DATASET :

Tuberculosis identification in microscopic sputum images has received little attention, despite the fact that we use realistic laboratory data samples. Those records are no longer available.

The COSTA data set comprises Tuberculosis bacilli microscopic sputum images. The objects in the TB dataset image are divided into three categories: single bacilli, touching or overlapping bacilli, and debris. Despite the fact that contacting bacilli are made up of multiple single bacillus objects that are touched together in various ways, their presence is critical in defining the inflexibility of the tuberculosis disease.
We classify the database into two categories.

- Bacilli
- Non-bacilli

The proposed system is implemented in two stages. In the first stage, we employed a segmentation method to classify the photos' foreground and background. In the foreground, there are single bacilli, touching bacilli, and other objects. The segmented foreground items are also sent into a trained convolutional neural network (CNN), which classifies them as bacilli or non-bacilli.

For training and testing the proposed CNN, we cropped positive patches and negative patches from images with different backgrounds from the aforementioned database.Sample positive and negative patches are shown in Fig. 3.

(a)

(b)

Fig (4.5) Sample patch images (a) positive patches (b) negative patches

patches, 80% were used for training and 20% were used as test sets. For validation, 20% of images from the training set were used.

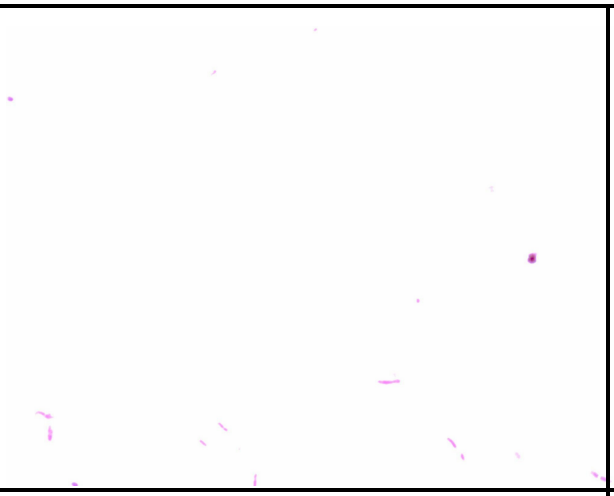
CHAPTER V
RESULT AND ANALYSIS

The image preprocessing technique and segmentation process used are able to remove background objects and highlight the foreground object, thereby increasing the chance of detecting bacilli much more effectively by the model. The image preprocessing used is contrast enhancement and able to retain the bacilli features. Masked-Simple linear iterative clustering is used for segmentation and is able to segment the bacilli efficently. The trained CNN model is used to predict the bacilli, for that a patch cropped from the segmented image is used. The proposed model achieved an accuracy of 94.62% and validation accuracy of 93.05%. This method is able to segment single bacilli from the background with less preprocessing technique and hence detection of TB bacilli became much easier.

Accuracy : 94.62%

Test Accuracy : 91.29%

Validation Accuracy : 93.05%

	b) Segmented image after preprocessing
	c) Cropped patch image from the fig.(b) for prediction Detected as bacilli

CHAPTER VI
CONCLUSION

Tuberculosis is a contagious disease for which early detection is important for effective treatment. The most common approach used in the routine diagnosis of tuberculosis is conventional microscopy. Tubercle bacilli are frequently identified in sputum smears using a microscope with manual detection in this procedure, which has been widely utilized in bacteriological investigation of sputum smears taken from suspected patients.

We were able to develop an automatic TB detection approach based on CNN employing microscopic sputum smear images in this project. These techniques can be integrated into an automated microscope to identify tuberculosis disease more accurately and quickly than manual detection. Patched bacilli pictures are used in the model. The obtained image is then given as input to a custom CNN for detection of TB. The result indicates whether the patient has TB or not.

CHAPTER VII
SCOPE FOR FUTURE DEVELOPMENT

There are other modifications that might be investigated to increase the model's scalability and accuracy. The system's performance can be improved by focusing more on denoising and boosting image resolution to high quality, which can lead to another favorable outcome. Only a single bacillus can be detected by the current technique. As a result, we may tweak the system to detect many bacilli from a single image. We can also use a more complex and appropriate segmentation algorithm that can detect dark areas in the image.

In the future, we can implement web based or mobile based applications using this project as base. Also, the design can be improved for reducing the time taken for execution of our code and also more parameters can be calculated. In future more other filters can also be included from the current proposed system of the project.

CHAPTER VIII

BIBLIOGRAPHY

[1] Automatic classification of light field smear microscopy patches using Convolutional Neural Networks for identifying Mycobacterium Tuberculosis by Yadini perez Lopez, C.F.F Costa Filho, L.M.R Aguilera, M.G.F Costa

[2] Towards Automated Tuberculosis detection using Deep Learning by Sonaal Kant, Muktabh Mayank Srivastava

[3] Automatic detection of tuberculosis bacilli from microscopic sputum smear images using deep learning methods by Rani Oomman Panicker, Kaushik S. Kalmady, Jeny Rajan, M.K. Sabu (2018)

[4] Automatic Detection and Classification of Tuberculosis Bacilli from ZN-Stained sputum smear images using watershed segmentation by Mohammad I Shah, Smriti Mishra, Malay Sarkar, Chittaranjan Rout

[5] Image preprocessing to improve Acid-Fast Bacilli (AFB) detection in smear microscopy to diagnose pulmonary tuberculosis by Jorge Luis Díaz-Huerta, Adriana del Carmen Téllez-Anguiano, José Antonio Gutiérrez-Gnecchi, Owen Yair Colin- González, Fanny Lucía Zavala-Santoyo, Sergio Arellano-Calderón (2019)

[6] Diagnosis and Counting of Tuberculosis Bacilli Using Digital Image Processing by Yoges Payasi and Savitanandan Patidar (2017)

[7] Automated identification of mycobacterium bacilli from sputum images for tuberculosis diagnosis by K. S. Mithra, W. R. Sam Emmanuel (2019)

[8]Analysis and comparison of image processing and artificial intelligence
algorithms to detect AFB in pulmonary tuberculosis images by Jorge L. Díaz-Huerta , Adriana C. T´ellez-Anguiano , Jos´e A. Guti´errez-Gnecchi a,Sergio Arellano-Calderon´ , Juan C. Olivares-Rojas (2022)

[9] Automated Tuberculosis Screening using Zeihl Neelson Image by Lavanya Govindan, Padmasini.N, Mohamed Yacin (2015)

[10] Research on Preprocessing Method for Microscopic Image of Sputum Smear and Intelligent Counting for Tubercule Bacillus by Zhisong Li, Jiayao Ling, Jing Wu, Nian Luo, Min Tan2 and Ping Zhong (2018)

[11] Image processing techniques for identifying Mycobacterium tuberculosis in Ziehl-Neelsen stains by P. Sadaphal, J. Rao,G. W. Comstock, M. F. Beg (2008)

CHAPTER IX

APPENDIX

a) Preprocessing

Contrast Enhancement

In [3]:
```
## contrast enhancement
```

In [4]:
```
def contrast(image):
    # Top Hat Transform
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(20,20))
    topHat = cv2.morphologyEx(image, cv2.MORPH_TOPHAT, kernel)
    # Black Hat Transform
    blackHat = cv2.morphologyEx(image, cv2.MORPH_BLACKHAT, kernel)
    contrast_image = image + topHat - blackHat
    contrast_image = cv2.cvtColor(contrast_image, cv2.COLOR_BGR2RGB)
    #plt.imshow(contrast_image)

    #cv2.imwrite('Process_image/contrast.jpg',contrast_image)
    return contrast_image
```

b) Mask slic

In [5]:
```
def mSlic(image):
    image = color.rgb2gray(image)
    mask = morphology.remove_small_holes(morphology.remove_small_objects(image < 0.5, 200),350)
    mask = morphology.opening(mask, morphology.disk(3))
    # maskSLIC result
    m_slic = segmentation.slic(image, n_segments=50, mask=mask, start_label=1)
    m_slic_label = segmentation.mark_boundaries(image, m_slic)
    m_slic= m_slic*255
    inverted_mask = np.invert(mask)
    #plt.imshow(m_slic)
    cv2.imwrite('process_image/mask_label',m_slic_label)
    cv2.imwrite('Process_image/mask.jpg',m_slic)

    return m_slic
```

c) CNN Model

Build a CNN model

In [57]:

```
cnnModel= models.Sequential([

    #first layer
    layers.Conv2D(3,(5,5) ,activation='relu', padding = 'same',input_shape=(224,224,3)),
    layers.Conv2D(32,3,activation='relu',),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #second layer
    layers.Conv2D(32,3,activation='relu'),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #third layer
    layers.Conv2D(64,3,activation='relu'),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #fourth layer
    layers.Conv2D(64,3,activation='relu'),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #fifth layer
    layers.Conv2D(64,3,activation='relu'),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #sixth layer
    layers.Conv2D(128,3,activation='relu'),
    layers.BatchNormalization(axis=-1),
    layers.MaxPooling2D((2,2)),

    #dense layer
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

d) Training

In [29]:
```
## Fitting the model
```

In [37]:
```
history=cnnModel.fit(X_train,y_train,
            epochs=25,
            batch_size=5,
            verbose=1,
            validation_split=0.2,
            callbacks=callbacks_list)
```

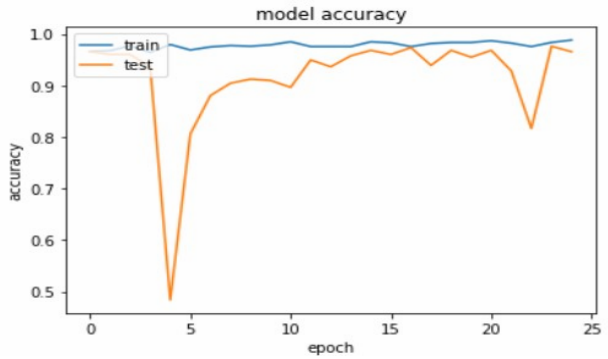
```
Train on 2240 samples, validate on 560 samples
Epoch 1/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.1411 - acc: 0.9485
Epoch 00001: val_acc did not improve from 0.96607
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.1414 - acc: 0.9482 - val_loss: 0.1427 - val_acc: 0.9500
Epoch 2/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.1072 - acc: 0.9709
Epoch 00002: val_acc did not improve from 0.96607
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.1070 - acc: 0.9710 - val_loss: 0.5718 - val_acc: 0.8179
Epoch 3/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.0749 - acc: 0.9812
Epoch 00003: val_acc did not improve from 0.96607
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.0748 - acc: 0.9812 - val_loss: 0.1355 - val_acc: 0.9625
Epoch 4/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.0772 - acc: 0.9776
Epoch 00004: val_acc improved from 0.96607 to 0.97321, saving model to saved_models/weights-improvement-04-0.97.hdf5
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.0770 - acc: 0.9777 - val_loss: 0.0969 - val_acc: 0.9732
Epoch 5/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.0782 - acc: 0.9754
Epoch 00005: val_acc did not improve from 0.97321
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.0781 - acc: 0.9754 - val_loss: 0.3905 - val_acc: 0.8571
Epoch 6/25
2235/2240 [============================>.] - ETA: 0s - loss: 0.0779 - acc: 0.9767
Epoch 00006: val_acc did not improve from 0.97321
2240/2240 [==============================] - 72s 32ms/sample - loss: 0.0778 - acc: 0.9768 - val_loss: 0.3097 - val_acc: 0.8786
```

e) Plotting Accuracy

Plotting Accuracy and Loss

In [57]:
```
# summarize history for accuracy

plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```



f) Plotting Loss

In [58]:
```
# summarize history for loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

